

Haskell Wikibook 中级专题

结合 EDA 的选题与一周执行方案

面向 4 人小组 presentation 的 20 页左右策划稿

推荐最终方向

Functional EDA: Using Haskell to Model and Simulate Boolean Circuits。以 Wikibook 的代数数据类型、类型类、Maybe/Either、State Monad、解析器组合子为基础，扩展到布尔电路建模、仿真、简单网表生成。

使用场景：用于小组内部确定选题、安排分工、准备汇报逻辑，也可作为和老师沟通选题可行性的说明材料。

时间约束：仅 1 周，因此方案采用“核心可交付 + 可选增强”的路线，保证即使增强部分来不及，也有完整 presentation。

1. 一句话结论

先把选择说清楚，再决定写多少代码。

建议选题

用 Haskell 构建极简组合逻辑 EDA 前端：从布尔表达式到 AST、真值表仿真与简单网表。

这个方向的优点是：它不是脱离课程的纯理论专题，也不是过大的工程项目。它能自然承接 Wikibook 的中级章节，同时又能延伸到 EDA 里的真实问题：电路如何表示、如何解析、如何仿真、如何转换成更接近网表的形式。

对一周准备周期来说，最稳妥的范围是：实现一个小型布尔表达式语言，手写或解析得到 AST，然后根据输入变量计算输出，最后打印真值表。若时间充足，再加入 State Monad 自动生成中间 wire 名称，展示从表达式到 netlist 的转换。

汇报时不需要把 Haskell 所有高级概念都讲完。你们要讲的是：为什么 Haskell 很适合表达电路结构，以及 Wikibook 中级专题如何在 EDA 场景里变成可运行的小工具。

项目	建议
最终标题	函数式 EDA：用 Haskell 表示与仿真布尔电路
核心 Wikibook 专题	代数数据类型、Type Classes、Maybe/Either、State Monad、Parser Combinators
核心 demo	输入一个布尔表达式，输出 AST、真值表、可选 netlist
难度定位	中级偏上，但核心 demo 可控

2. 老师要求的真实含义

不是照读 Wikibook，而是从专题出发做扩展。

老师说 Wikibook 里有很多中级难度专题，建议从中查找相关专题进行扩展学习和讲解。这里的关键词不是“复述”，而是“扩展”。也就是说，小组需要先选一个 Wikibook 中确实存在的中级主题，再说明这个主题可以解决什么问题，以及在一个更具体的应用场景中如何落地。

如果只是讲 Monad、Functor 或 Type Classes，听起来会像概念介绍，容易抽象、容易散。把它们放进 EDA 场景之后，主题就会变成：Haskell 的类型系统和函数式组合方式，如何帮助我们描述和处理电路结构。这样选题会更像一次真正的扩展学习。

老师要求	我们的对应策略
中级难度	选 Wikibook Intermediate Haskell 之后的专题，不选入门语法
专题扩展	不只讲概念，扩展到 EDA 中的布尔电路前端
小组 presentation	分成背景、建模、仿真、扩展四块，四人都能讲
一周完成	核心 demo 必须小；增强功能只作为可选项

和老师沟通时可以这样说

我们打算以 Wikibook 的中级专题为基础，选择代数数据类型、类型类和 Monad 相关内容，扩展到 EDA 中的布尔电路表示与仿真，做一个极简函数式 EDA 前端示例。

3. 选题评价标准

一周项目要看“能讲清楚”和“能做出来”。

选题不要只看听起来高级，还要看能不能在一周内变成清晰的演示。一个好的 presentation 选题至少满足五个条件：和 Wikibook 章节有明确对应；和 EDA 有自然连接；代码 demo 不依赖复杂环境；四个人能拆开准备；即使某个增强功能没做完，主线仍然完整。

标准	含义	本方案表现
Wikibook 对应	能指向明确章节	ADT、Type Classes、Functor/Monad、State、Parser 都有章节
EDA 关联	不是硬凑概念	布尔表达式、AST、真值表、netlist 都是 EDA 前端常见元素
可演示性	能运行小程序	核心代码约 100-200 行即可完成
分工性	四个人都有独立任务	背景、建模、求值、转换/解析可拆开
风险可控	一周内能收敛	采用核心功能优先、增强功能可砍策略

最重要的判断是：不要把最终目标定成“实现一个 EDA 工具”。那太大。应该定成“用 Haskell 的中级专题解释 EDA 前端中的一个简化问题”。这样老师看到的是专题扩展，而不是超出能力范围的工程承诺。

4. Wikibook 可选专题地图

下面这些都能和 EDA 发生联系，但工作量不同。

Wikibook 专题	可扩展到 EDA 的角度	难度	建议
More on datatypes / Trees	用 ADT/树表示电路和表达式	中	必须选，主线基础
Classes and types	给表达式定义 pretty print、eval 等行为	中	适合作为讲解亮点
Functor / Maybe	处理可能失败的变量查询、解析失败	中	适合讲错误处理
State Monad	自动生成 wire 名称、节点编号	中偏上	推荐作为增强
Parser Combinators	把文本表达式解析成 AST	中偏上	展示效果好，但可选
Testing / QuickCheck	随机生成输入验证电路等价性	中偏上	作为结尾展望即可

因此最合理的组合不是选单个孤立概念，而是让它们形成一条链：ADT 表示电路，Either/Maybe 处理错误，State 生成网表，Parser 把字符串变成 AST。汇报时可以把 ADT 作为核心，其余概念按时间选择性展开。

优先级

必做：ADT + evaluator + truth table。推荐做：State Monad 生成 netlist。可选做：Parser Combinators。

只展望：QuickCheck 等价性测试。

5. 为什么 EDA 适合和 Haskell 结合

EDA 的很多对象本来就是“结构 + 变换”。

EDA 不是只有画电路图和跑商业软件。站在软件角度看，EDA 中有大量任务都可以抽象成数据结构和变换：HDL 源码被解析成语法树，语法树被 elaboration 成 netlist，netlist 再被优化、仿真、布局布线。Haskell 擅长的正是代数数据类型、递归结构、纯函数变换和类型约束。

EDA 概念	Haskell 对应思想	可在 demo 中体现
布尔表达式	递归 ADT	<code>Expr = Var Not And Or</code>
电路仿真	纯函数求值	<code>eval env expr</code>
未知信号/错误	Maybe / Either	变量不存在时返回错误
中间节点编号	State Monad	自动生成 n1, n2, n3
HDL 前端解析	Parser Combinators	字符串到 AST
逻辑优化	模式匹配重写	<code>Not (Not x) => x</code>

这就是本选题的说服力：Haskell 不是被强行拉到 EDA 里，而是它的语言特性刚好适合表达 EDA 前端的核心动作。

6. 推荐题目定义

题目要小，但要能看到完整流程。

正式题目

函数式 EDA：用 Haskell 表示与仿真布尔电路

建议把 presentation 的核心问题定义为：给定一个简单的组合逻辑表达式，如何用 Haskell 把它表示成数据结构，并进一步进行求值、真值表生成和简单网表转换。这个问题足够小，不需要涉及时序逻辑、Verilog 完整语法、优化算法或物理设计；但它又足够像 EDA，因为它包含“前端表示”和“语义计算”。

范围	做什么	不做什么
输入	布尔表达式，如 (a AND b) OR (NOT c)	不解析完整 Verilog
内部表示	AST 表达式树	不建复杂图数据库
输出	真值表，简单 netlist	不做综合、布局布线
理论连接	ADT、Type Classes、Monad、Parser	不讲完整范畴论

这样定义后，汇报的主线非常清楚：从一个 EDA 问题出发，展示 Haskell 中级概念如何一步步支撑这个问题。

7. Demo 的最小闭环

最小闭环比“大而全”更重要。

一周内最需要保证的是 demo 有闭环。建议核心 demo 固定为三步：构造表达式 AST；对所有输入组合求值；打印真值表。这个闭环不依赖第三方库，即使 parser 没做完，也可以手写 AST 作为输入。

```
-- example circuit:
-- out = (a AND b) OR (NOT c)

expr :: Expr
expr = Or (And (Var "a") (Var "b")) (Not (Var "c"))

-- expected presentation output:
-- a b c | out
-- 0 0 0 | 1
-- 0 0 1 | 0
-- ...
```

层级	功能	完成标准
Core	ADT + eval + truth table	必须能运行，汇报现场可演示
Plus	表达式 pretty print + 简单化简	让输出更像工具
Advanced	Parser 或 State netlist	二选一，做成亮点

底线策略
不要把 parser 设为唯一入口。现场演示时允许使用手写 AST，这样解析器出问题也不会毁掉主线。

8. Haskell 建模：用 ADT 表示电路

这部分由第二位同学重点负责。

代数数据类型是本题最核心、最容易讲清楚的 Haskell 中级内容。电路表达式天然递归树：变量是叶子节点，NOT 是一元节点，AND/OR 是二元节点。Haskell 的 data 声明可以非常直接地表达这种结构。

```
data Expr
  = Var String
  | Not Expr
  | And Expr Expr
  | Or Expr Expr
  deriving (Eq, Show)
```

这个定义可以用来解释两个点：第一，ADT 让我们直接定义领域模型，而不是先用字符串或数组硬凑；第二，递归数据结构和模式匹配非常适合描述电路树的遍历、求值、打印和转换。

构造器	EDA 含义	例子
Var String	输入信号或 wire	Var "a"
Not Expr	反相器	Not (Var "c")
And Expr Expr	与门	And (Var "a") (Var "b")
Or Expr Expr	或门	Or x y

9. 求值与错误处理

让 Maybe/Either 变成有意义的 EDA 语境。

电路仿真需要一个环境 env，把信号名映射到布尔值。如果表达式中出现了 env 中没有的变量，程序不应该随便给默认值，而应该返回错误。这正好可以引出 Wikibook 中 Maybe/Either 和 Monad 的作用：把“可能失败的计算”显式写进类型。

```
type Env = Map String Bool
```

```
eval :: Env -> Expr -> Either String Bool
eval env (Var x) =
  maybe (Left ("unknown signal: " ++ x)) Right (Map.lookup x env)
eval env (Not e)  = not <$> eval env e
eval env (And a b) = (&&) <$> eval env a <*> eval env b
eval env (Or  a b) = (||) <$> eval env a <*> eval env b
```

这段代码可以讲出 Haskell 的几个特色：类型签名说明函数可能失败；模式匹配按电路结构递归；

Applicative 写法把两个子表达式的计算组合起来。听众不需要掌握所有抽象细节，也能理解它解决的问题。

讲解重点

不要先讲抽象 Monad 定义。先讲 EDA 中“未知信号”的问题，再展示 Either 如何把错误传播变得清楚。

10. 真值表生成

这是最适合现场展示的输出。

真值表是数字逻辑课程中熟悉的形式，能降低听众理解门槛。技术上，只要收集表达式中的变量名，枚举所有布尔输入组合，再调用 eval 即可。这个过程能展示 Haskell 的 list processing、map、filter、fold 等内容。

```
vars :: Expr -> [String]
vars (Var x)    = [x]
vars (Not e)    = vars e
vars (And a b)  = nub (vars a ++ vars b)
vars (Or  a b)  = nub (vars a ++ vars b)

assignments :: [String] -> [Env]
assignments []  = [Map.empty]
assignments (v:vs) =
  [ Map.insert v b env | b <- [False, True], env <- assignments vs ]
```

输入 a	输入 b	输入 c	out = (a AND b) OR NOT c
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	1	0	1
1	1	1	1

11. Type Classes 的扩展讲法

把“接口”讲成电路工具行为。

如果只讲 Type Classes，可能会显得和 EDA 没关系。但在这个题里，可以把它讲成“同一个电路表达式可以有多种行为”：显示、求值、统计门数量、生成 netlist。Type class 就像给不同数据类型定义一组共同能力。

```
class Pretty a where
  pretty :: a -> String

instance Pretty Expr where
  pretty (Var x)    = x
  pretty (Not e)    = "NOT " ++ pretty e
  pretty (And a b) = "(" ++ pretty a ++ " AND " ++ pretty b ++ ")"
  pretty (Or a b)  = "(" ++ pretty a ++ " OR " ++ pretty b ++ ")"
```

可定义的行为	含义	展示方式
Pretty	把 AST 打印成人可读表达式	pretty expr
Eval	在输入环境中计算输出	eval env expr
GateCount	统计门数量	countGates expr
Netlistable	生成简单网表	toNetlist expr

这部分不必写太多代码。presentation 中用一页说明 Type Classes 如何把“能力”抽象出来即可。

12. State Monad：从表达式到网表

这是最推荐的增强功能。

EDA 中的 netlist 通常会引入中间节点。例如 (a AND b) OR (NOT c) 可以转换为 n1 = AND a b, n2 = NOT c, out = OR n1 n2。这里有一个小问题：中间 wire 名字需要自动编号。State Monad 正好适合管理这种“计数器状态”。

```
-- idea only:
fresh :: State Int String
fresh = do
  n <- get
  put (n + 1)
  return ("n" ++ show n)

-- output:
-- n1 = AND a b
-- n2 = NOT c
-- out = OR n1 n2
```

普通做法	State Monad 做法
手动传入当前编号，并返回新编号	把编号状态封装在 State 中
函数参数越来越多	转换函数保持清晰
容易忘记更新计数器	fresh 统一生成名字

为什么它适合汇报

State Monad 比抽象 Monad 更容易讲，因为它有具体任务：自动生成 wire 名称。听众能直接看到它在 EDA 场景中的价值。

13. Parser Combinators: 可选但很亮眼

如果时间紧，可以只讲思路，不强求完整实现。

解析器组合子可以把文本表达式转换成 AST，类似一个极简 HDL 前端。如果实现顺利，它会让 demo 更像一个工具：用户输入字符串，程序输出真值表和 netlist。但 parser 的边界处理比较花时间，因此应作为可选增强。

解析层级	例子	输出
变量	a	Var "a"
一元操作	NOT c	Not (Var "c")
二元操作	a AND b	And (Var "a") (Var "b")
括号表达式	(a AND b) OR NOT c	Or (And ...) (Not ...)

如果你们没有时间写完整 parser，可以在汇报中用伪代码说明“Parser Expr 是一个从字符串到 Expr 的函数”，再展示手写 AST 的核心 demo。这样 parser 仍然是专题扩展，但不会成为项目风险。

取舍建议

四个人一周：State Monad netlist 比完整 parser 更稳。若组内有人很熟 Haskell，再做 parser。

14. 三档选题方案

根据实际进度随时降级，汇报仍然完整。

档位	内容范围	适合情况	风险
A 稳妥版	ADT + eval + truth table + Wikibook 对应讲解	时间紧、Haskell 基础一般	展示稍朴素，但完整
B 推荐版	A + State Monad 生成简单 netlist	最平衡，既稳又有亮点	需要多写一段转换代码
C 冲刺版	B + Parser Combinators 解析输入字符串	组内有人愿意主攻 parser	调试时间可能超出预期

建议你们把对外承诺定为 B 推荐版：核心完成后有明显 EDA 味道，而且 State Monad 和 Wikibook 中级专题对应明确。Parser 放在“如果时间允许”的扩展里，避免被它拖住。

最终汇报中必须出现	可以根据时间删减
AST 数据结构、求值函数、真值表、分工说明	完整 parser、QuickCheck、复杂逻辑优化、GUI

15. 四人分工

每个人既有讲解内容，也有可交付物。

成员	负责模块	具体产出	汇报页数
A	背景与选题动机	EDA 前端流程图、Wikibook 章节对应、选题价值说明	3-4 页
B	Haskell 建模	Expr ADT、模式匹配、Pretty/Show 展示	4-5 页
C	仿真与错误处理	eval、Either/Maybe、真值表生成、现场 demo 主流程	4-5 页
D	扩展与整合	State Monad netlist；可选 parser；整合代码与演示脚本	4-5 页

这个分工的好处是每个人的内容都能独立准备，但最后又能合成一条完整故事线。A 负责“为什么选这个题”，B 负责“如何表示电路”，C 负责“如何仿真”，D 负责“如何接近 EDA 工具”。

避免不均衡
不要让一个人单独承担所有代码。B 写数据结构，C 写求值和真值表，D 写 netlist 或 parser，A 负责汇报叙事和资料整合。

16. 一周时间表

按可交付物推进，不按“读完再做”推进。

日期	目标	产出
Day 1	确定题目和范围	题目一句话、功能边界、四人分工、代码仓库/文件夹
Day 2	完成 AST 与求值	Expr 定义、eval 函数、2-3 个测试表达式
Day 3	完成真值表与讲解初稿	truth table 输出、前 8-10 页 slides 草稿
Day 4	完成 State netlist 或 parser 二选一	可运行增强功能；若卡住则保留伪代码
Day 5	整合 slides 和 demo	完整 slides 初版、演示脚本、失败备用路径
Day 6	第一次正式彩排	计时、删减、修正讲解顺序、截图备份
Day 7	最终检查	最终版 PPT/PDF、代码压缩包、现场讲稿

这份排期的核心是 Day 3 之前必须出现可运行的核心 demo。不要等到最后两天才写代码。后面几天主要用于增强、整合和排练。

17. Presentation 结构建议

20 分钟左右可以这样分配。

时间	内容	负责人
0-3 min	问题引入：EDA 前端为什么需要表示、解析和仿真	A
3-7 min	Wikibook 中级专题对应：ADT、Type Classes、Monad	A/B
7-11 min	用 ADT 建模布尔电路，展示 AST	B
11-15 min	求值、错误处理和真值表 demo	C
15-18 min	State Monad 生成 netlist 或 parser 扩展	D
18-20 min	总结：函数式思想对 EDA 的价值与局限	A/D

PPT 页数建议控制在 16-22 页。代码页不要太多，每页只放一个关键片段，并配合运行输出。老师更关心你们是否理解专题和扩展逻辑，不会要求看到完整工程细节。

现场演示建议

准备两种演示：一种是现场运行；另一种是截图或录屏。如果 Haskell 环境临时出问题，仍然可以讲完整。

18. 风险与降级方案

一周项目的关键不是无风险，而是知道怎么降级。

风险	表现	处理办法
Parser 写不完	括号、优先级、空格处理出错	删掉完整 parser，用手写 AST；parser 只作为思路讲
State Monad 难讲	听众不理解抽象	只讲 wire 编号问题，不讲复杂 Monad 理论
代码现场跑不起来	环境路径、包依赖出错	提前准备输出截图和录屏
内容太散	Haskell 概念堆太多	所有概念都回到同一个表达式 demo
四人衔接断	每人像讲不同题	统一用“表达式到真值表/网表”作为主线

降级时不要觉得丢分。一个清晰、可运行、讲得懂的 A 版，比一个 parser 半成品但讲不清的 C 版更适合课堂 presentation。

硬性里程碑

Day 3 晚上如果核心 demo 还不能运行，立刻停止增强功能，全组转向完成 AST + eval + truth table。

19. 可以直接使用的选题说明

这段可以放进开题说明或发给老师。

我们小组计划选择“函数式 EDA：用 Haskell 表示与仿真布尔电路”作为 presentation 主题。该主题以 Wikibook 中 Intermediate Haskell 部分的代数数据类型、类型类、Maybe/Either、State Monad 和解析器组合子等内容为基础，扩展到 EDA 中的组合逻辑表达式建模与仿真问题。

具体来说，我们会用 Haskell 的 ADT 定义一个极简布尔表达式语言，包括变量、NOT、AND、OR 等构造；再通过模式匹配和递归函数对表达式进行求值，生成真值表；进一步地，我们会尝试用 State Monad 自动生成中间 wire 名称，把表达式转换为简单 netlist。若时间允许，还会展示如何用 parser combinators 将文本表达式解析成 AST。

这个选题的重点不是实现完整 EDA 软件，而是通过一个小而完整的例子说明：Haskell 的类型系统、递归数据结构和函数式组合思想，如何用于表达和处理电路设计中的前端问题。

核心承诺	保证完成
扩展承诺	State netlist 尽量完成，parser 视时间决定
展示价值	把抽象的 Haskell 中级专题放入 EDA 应用场景

20. 最终检查清单

用这个清单判断你们是否已经准备好。

检查项	完成标准
题目是否清楚	一句话能说出：用 Haskell 做极简布尔电路建模与仿真
Wikibook 对应是否明确	至少能指出 ADT、Type Classes、Maybe/Either、State Monad 等章节
EDA 关联是否自然	能解释 AST、真值表、netlist 与 EDA 前端的关系
核心 demo 是否可运行	同一表达式能输出真值表；错误变量能返回清楚提示
四人是否都有产出	每人 3-5 页内容或一个独立代码/讲解模块
是否有备用方案	有截图、录屏或预生成输出，避免现场环境问题
时间是否可控	20 分钟讲完，代码展示不超过 5 分钟

最终建议

把项目定位成“中级 Haskell 专题在 EDA 前端中的一次小型应用”。这比单讲 Monad 更具体，比做完整 EDA 工具更可控，也更适合四个人一周完成。

如果老师允许，你们还可以在汇报最后用一句话扩展到更大的方向：同样的思想可以继续用于 HDL 解析、逻辑优化、电路等价性检查、随机测试和硬件生成 DSL。这样既能展示眼界，也不会增加当前实现负担。